PROTOCOL

Technical Whitepaper

Proof of Useful Work (PoUW) Consensus Protocol in the zk-Tensor Core Architecture

Abstract

This document specifies the technical architecture of the PROTOCOL ecosystem. It defines the mathematical foundations of the Proof of Useful Work (PoUW) consensus, built on an optimized linear-algebra engine (the zk-Tensor Core). The system implements error-free arithmetic decomposition into INT8 format, cryptographic tensor fragmentation (Matrix Tiling), exact precision aggregation, and commitment-bound probabilistic verification via the Stochastic Matrix Audit (SMA) algorithm. The architecture is complemented by a scalable data-availability layer with transparent polynomial commitments (P-DAL + FRI) and a private, post-quantum transaction layer (zk-TWL).

Contents

1 System Architecture and Base Parameters

PROTOCOL relies on a deterministic monetary policy. Token emission follows a deflationary curve, which stabilizes the profitability of the computational infrastructure while maintaining a fixed maximum supply.

1.1 Ecosystem Parameters Table

Component	Technical Description
Maximum Supply ($S_{\max}$)	Asymptotically approaches 15,000,000 [TKN].
Genesis Allocation	2,250,000 [TKN] allocated in Block 0, intended for development, incentive mechanisms, and the testnet.
Emission Model	10% annual decrease ($\rho = 0.10$).
Algorithmic Consensus	Proof of Useful Work (PoUW) optimized for linear algebra, based on INT8 arithmetic with exact floating-point reconstruction and the SMA audit.
Entry Barrier and Penalties	No capital stake required (0 TKN). Sybil resistance derives from the cost of verified work (Pay-Per-Share model): every identity must independently perform and pass validation of its own shares. Penalties rely on block rejection and Quarantine.
Transaction Privacy	zk-TWL (zk-STARK proofs and CRYSTALS-Dilithium signatures) preserving full transfer confidentiality.

Table 1: Ecosystem Parameters.

1.2 Supply Function (Smooth Emission)

Emission reduction is modeled as a geometric progression. The total supply in year t is:

$$S_{\text{current}}(t) = G + \sum_{n=0}^{t} \overline{R}_0(1-\rho)^n = G + \overline{R}_0 \frac{1-(1-\rho)^{t+1}}{\rho} \tag{1}$$

The asymptotic limit defines the fixed maximum supply:

$$S_{\max} = \lim_{t\to\infty} S_{\text{current}}(t) = G + \frac{R_0}{\rho} = 2,250,000 + \frac{1,275,000}{0.10} = 15,000,000\,[\text{TKN}] \tag{2}$$

Legend:

- G: Genesis Allocation generated in Block 0 (2,250,000 [TKN]).
- $S_{\text{current}}(t)$: Integrated token supply in the network in year t.
- R_0: Base reward in the first year (Genesis Epoch), 1,275,000 [TKN].
- ρ: Annual emission reduction rate, 0.10.
- t: Year index since network initialization.

1.3 Internal Fee Market

The internal pricing of a computational unit is dynamically calibrated by market mechanisms:

$$P_{\text{compute}} = \frac{Fee_{\text{task}}}{N_{\text{ops}}} \tag{3}$$

Legend:

- P_{compute}: Profit per elementary computational unit, in [TKN].
- Fee_{task}: Total task budget, in [TKN].
- N_{ops}: Estimated volume of computational operations (a function of matrix size).

2 Computational Layer: zk-Tensor Core

The network engine executes mathematical operations using the zk-Tensor Core architecture, optimized for large-scale tasks.

2.1 Generalized Tensor Equation (GTE) and Zero-Padding

Every problem introduced to the network is reduced to a Generalized Tensor Equation (GTE):

$$A \times B + \Omega = C \tag{4}$$

The engine operates over a configurable algebraic domain: over a finite ring for cryptographic tasks ($A \times B + \Omega \equiv C \pmod{p^k}$), or over floating-point numbers for numerical tasks, where product exactness is guaranteed by the error-free decomposition (Sec. 2.2).

For asymmetric or irregular matrices, the network automatically applies the Zero-Padding technique, virtually expanding the data edges. This allows lossless task division without affecting the final mathematical result.

Application	**Matrix A**	**Matrix B**	**Noise/Shift Ω**	**Result C**
Artificial Intelligence	Network Weights (W)	Input Data (X)	Bias Vector	Activations
Fluid Mechanics	Stiffness Matrix (K)	System State (U)	Force Vector (F)	Balance
Lattice Cryptography	Lattice Basis (A)	Secret Key (S)	Gaussian Noise (E)	Public Key

Table 2: Implementations of the Generalized Tensor Equation.

2.2 Error-Free Decomposition and Cryptographic Tensor Fragmentation

To let researchers maintain high floating-point precision without burdening the silicon, the network implements an error-free arithmetic decomposition. Instead of statistical noise averaging, each floating-point matrix is decomposed into a sum of bounded-width integer slices whose products are computed exactly in INT8 arithmetic with INT32 accumulation.

For a matrix $A \in \mathbb{R}^{M \times K}$ we select a slice width of b bits and decompose:

$$A = \sum_{i=1}^{s_A} 2^{-\sigma_i^A} A^{(i)}, \quad A^{(i)} \in \mathbb{Z}^{M \times K}, \quad |A_{kl}^{(i)}| < 2^b \tag{5}$$

where σ_i^A is the scale exponent of slice i, and the number of slices $s_A = \lceil \mu / b \rceil$ depends on the requested mantissa precision μ (in bits). Dynamic range is handled by per-tile scaling. Matrix B is decomposed analogously.

By linearity, the full product is reconstructed from exact integer partial products:

$$AB = \sum_{i=1}^{s_A} \sum_{j=1}^{s_B} 2^{-(\sigma_i^A + \sigma_j^B)} A^{(i)} B^{(j)} \tag{6}$$

Each partial product $A^{(i)}B^{(j)}$ is computed on the tensor core in INT8 $\rightarrow$ INT32 format with no rounding. The no-overflow condition for the accumulator, for a tile of inner dimension n, is:

$$2b + \lceil \log_2 n \rceil \leq 31 \tag{7}$$

For INT8 operands ($b \leq 7$) and $n = 512$ we obtain $14 + 9 = 23 \leq 31$, with margin. Slice pairs whose combined scale falls below the target unit in the last place (ULP) are pruned, which reduces the number of products to a triangular subset of size $\approx s_A s_B/2$.

The no-overflow bound is a checkable invariant with headroom. The condition above is a protocol invariant, verified per configuration rather than assumed. At $b = 7$ it admits an inner tile dimension up to $n = 2^{31-14} = 2^{17}$ before INT32 saturates, so the reference parameters ($b = 7, n = 512$) sit far inside the safe region. Should a configuration ever approach the limit, the bound is restored either by finer slicing (a smaller b, which increases the slice count s) or by widening the accumulator to INT64; the choice is an efficiency trade-off, not a correctness one.

Target Precision	**Mantissa Bits μ**	**Slices $s = \lceil \mu/b \rceil$ ($b = 7$)**	**INT8 Products (after pruning)**
FP16	11	2	order of units
FP32	24	4	order of 10
FP64	53	8	order of tens

Table 3: Reconstruction cost: a fixed, deterministic quantity. For comparison, a noise-averaging approach would require on the order of 10^6 iterations for FP32 and 10^{24} for FP64.

To eliminate the hardware I/O bottleneck (the "Memory Wall"), gigantic input matrices $A \in \mathbb{R}^{M\times K}$ and $B \in \mathbb{R}^{K\times N}$ are partitioned into standardized sub-matrices (tiles) of fixed block dimension $n \times n$ (e.g. $n = 512$). Formally, the inputs are represented as block structures:

$$A = [A_{i,k}]_{i=1,k=1}^{M',K'}, \quad B = [B_{k,j}]_{k=1,j=1}^{K',N'} \tag{8}$$

where $M' = \lceil M/n \rceil$, $K' = \lceil K/n \rceil$, and $N' = \lceil N/n \rceil$. A miner processes exclusively its assigned tile, producing a local sub-result $C_{i,j|k}$ (the tile fragment i, j originating from block k). That sub-result is itself the exact scaled sum of the bounded integer partial products $A_{i,k}^{(\cdot)} B_{k,j}^{(\cdot)}$ of the slice decomposition.

A claimed matrix product $XY = Z$ is verified deterministically in $\mathcal{O}(n^2)$ time rather than the $\mathcal{O}(n^3)$ cost of re-multiplication by drawing a challenge vector $\vec{h} \in \mathbb{F}_p^n$ over a finite field (its derivation and binding to the result are given in Sec. 2.4) and checking the identity:

$$X(Y\vec{h}) - Z\vec{h} = \vec{0} \pmod{p} \tag{9}$$

By the associativity of matrix multiplication the column vector $Y\vec{h}$ is evaluated first, which reduces the total cost of validation to $\mathcal{O}(n^2)$ and allows on-the-fly computation within the ultra-fast L1/L2 cache of the GPU. In PROTOCOL the audit is applied to each bounded integer partial product of the decomposition, i.e. $X = A^{(i)}$, $Y = B^{(j)}$, $Z = A^{(i)}B^{(j)}$.

Because the SMA identity is checked on the bounded integer partial products $A^{(i)}B^{(j)}$, every audited entry obeys the no-overflow bound and is therefore strictly smaller than the field modulus:

$$|[A^{(i)}B^{(j)}]_{kl}| < 2^{2b+\lceil \log_2 n \rceil} \leq 2^{31} \ll p = 2^{61} - 1 \tag{10}$$

Consequently, reduction modulo p acts as the identity on the audited quantities: a passing audit certifies exact equality over $\mathbb{Z}$, not merely congruence modulo p. The classical gap in which an adversary submits $C' = AB \pmod{p} \neq AB$ and passes with probability 1 cannot arise here,

because the audited values never reach p. The full, potentially large product is reconstructed only by exact summation in FP64, outside the field (Sec. 2.3), and is never itself reduced modulo p. An explicit range assertion on the revealed entries (cost $\mathcal{O}(n^2)$) may accompany the audit to make this integer-equivalence verifiable by inspection.

If $A^{(i)}B^{(j)} \neq Z$, then for $\vec{h}$ drawn uniformly from $\mathbb{F}_p^n$:

$$\Pr[(A^{(i)}B^{(j)} - Z)\vec{h} = \vec{0}] \leq \frac{1}{p} \tag{11}$$

A single challenge over $p = 2^{61} - 1$ thus bounds the cheating probability by $\approx 2^{-61}$; equivalently, an adversary grinding candidate results would need $\approx 2^{61}$ attempts below a comfortable cryptographic margin. The protocol therefore amplifies soundness by drawing r independent challenge vectors from the same commitment (Sec. 2.4):

$$\vec{h}^{(l)} = H(\kappa \parallel Block_{\text{prev}} \parallel l) \in \mathbb{F}_p^n, \quad l = 1, \ldots, r \tag{12}$$

which yields a bound of p^{-r}. With $r = 3$ this is $\approx 2^{-183}$ (grinding cost $\approx 2^{183}$, infeasible); $r = 2$ is already $\approx 2^{-122}$. We adopt $r = 3$ as the default, exceeding a 128-bit security margin with headroom. Each field element is obtained from the hash output by rejection sampling (or from a wider digest) to remove the slight non-uniformity of reducing a 64-bit value modulo $2^{61} - 1$; this bias is cryptographically negligible but is eliminated for cleanliness.

2.3 Precision Aggregation Protocol (Result Rollup)

When the network has validated and stored all component partial products, the aggregation layer reconstructs the floating-point result by an exact summation of the verified integer partial products with their corresponding scales:

$$C_{\text{float}} = \sum_{(i,j)\in P} 2^{-(\sigma_i^A + \sigma_j^B)} C_{\text{int}}^{(i,j)} \tag{13}$$

$$C_{\text{int}}^{(i,j)} = A^{(i)}B^{(j)} \tag{14}$$

Legend:

- P: The set of retained slice pairs (after pruning terms below the ULP).
- $C_{\text{int}}^{(i,j)}$: The exact INT32 partial product.
- σ_i^A, σ_j^B: The scale exponents of the slices.

The summation is performed in FP64. The result is exact down to the target ULP, with no statistical error and no dependence on a number of repetitions. The client receives a ready, precise result, maintaining full abstraction of the protocol's hardware operations.

2.4 Anti-Fraud Mechanism: Result Binding and Sequential Time Seal

To secure the system against pre-computation attacks and identifier manipulation (grinding), the audit uses a commit-reveal scheme (the Fiat-Shamir heuristic), and block finalization uses a Verifiable Delay Function (VDF).

1. Commitment: the miner publishes $\kappa = H(C \parallel TaskID \parallel MinerID)$ before the challenge is known.
2. Challenge derivation: the r challenge vectors are derived deterministically as $\vec{h}^{(l)} = H(\kappa \parallel Block_{\text{prev}} \parallel l) \in \mathbb{F}_p^n$ for $l = 1, \ldots, r$.

3. Reveal and verification: the miner reveals C; the node checks $H(C \parallel \dots) = \kappa$ and the r SMA equations.

Because each $\vec{h}^{(l)}$ is a deterministic function of the commitment to C, an adversary cannot construct $C \neq AB$ satisfying $(AB - C)\vec{h}^{(l)} = \vec{0}$; any change to C changes κ, and therefore every $\vec{h}^{(l)}$. This closes adaptive forgery, and grinding a passing commitment requires simultaneously defeating all r independent challenges, i.e. probability p^{-r}.

Sequential Time Seal (VDF):

$$(y, \pi) = VDF(x, T) \tag{15}$$

$$x = H(\kappa \parallel Block_{\text{prev}}) \tag{16}$$

The VDF requires T sequential (inherently non-parallelizable) steps and produces a proof π verifiable in $\mathcal{O}(\text{polylog } T)$ time. A block is valid only with a correct pair (y, π). The mechanism enforces a minimal, non-parallelizable time interval and removes the advantage of pre-computation and identifier grinding.

Classic VDF constructions (repeated squaring in groups of unknown order) are not post-quantum. If strict post-quantum resistance is required, the delay function can be realized as a sequential hash chain (hash-based assumptions), at the cost of a less succinct proof. Manipulation of timing alone is of limited value compared to forging computation, which is independently prevented by the binding scheme.

3 Consensus, Penalties and Economics (PoUW)

3.1 Sybil Resistance and Quarantine

Resistance to Sybil attacks derives from the cost of verified work, not from a hardware identity. In the Pay-Per-Share model, reward is proportional to validated work; creating multiple identities yields no advantage, because each one must independently perform the computation and pass validation of its own shares. The barrier is the real cost of computation and energy.

The MinerID is a cryptographic identifier derived from the node's public key. Reputation is an asset earned through correctly validated work and may gate access to higher-value tasks; Quarantine causes its forfeiture together with any unsettled rewards.

3.2 Automated Proportional Settlements (Pay-Per-Share)

Breaking massive tasks into tiles transforms profit distribution into a proportional model that pays for the actual amount of executed and validated work:

$$W_{\text{node}} = \left(\frac{v_{\text{tiles}}}{V_{\text{total}}} \right) \cdot Fee_{\text{task}}\,[\text{TKN}] + R_{\text{block}}(t) \tag{17}$$

3.3 Dual-Mode Architecture and Difficulty Regulator (PID)

The system operates in two modes, distinguished by a binary switch on the availability of paid tasks.

Commercial Mode: Difficulty is exogenous, imposed by the client's real task. The protocol does not regulate it; it only prices it through the fee market (Sec. 1.3).

Incident Type	Protocol Procedure	Consequence
Computational Error / Failure	Failure to meet the mathematical conditions of the SMA algorithm for the processed tile.	Share/block rejection. The node loses the energy (electricity) used for generation, with no reward allocation.
Forgery Attempt	Detection of result manipulation by the binding scheme (Sec. 2.4) or on-chain data forgery.	Activation of Quarantine (Timeout Jail). Forfeiture of unsettled rewards and of the accumulated reputation associated with the identifier.

Table 4: Security and Penalty Policy.

Synthetic Mode: Activated when the paid task pool is empty. Difficulty becomes endogenous and is steered by a PID regulator that maintains a target block time:

$$D(n) = D_{\text{base}} + K_p e(n) + K_i \sum_{k=0}^{n} e(k) + K_d[e(n) - e(n-1)] \tag{18}$$

The integral term eliminates steady-state error; the derivative term damps oscillations. In Synthetic Mode, the protocol allocates available capacity to public-good tasks: optimization of open models (Open Weights) and generation of cryptographic infrastructure (ZK proofs). This maintains the utility of the network outside the context of commercial tasks.

4 Transaction and Data Storage Layer (P-DAL)

PROTOCOL couples a data-availability layer for tensor structures with a dual-pool transaction model. A shielded pool provides confidential transfers (hash-based commitments, keyed nullifiers, zk-STARK validity); a transparent pool carries the public computational market (ML-DSA signatures). Every primitive is hash-based or lattice-based: no elliptic curves, no pairings, no trusted setup. Confidentiality-critical guarantees are carried by cryptography; correctness, conservation, anonymity-set size, and boundary bounds are carried by non-cryptographic mathematics, information theory, and mechanism design.

Global notation:

- H — STARK-friendly, collision-resistant hash, output in $\mathbb{F}_p^t$ ($t = 5$, so $p^t > 2^{256}$).
- $\langle\cdot\rangle$ — domain-separation tag prefixing each hash invocation.
- $\mathbb{F}_p$ — native field, $p = 2^{61} - 1$; $\mathbb{F}_{p^k}$ — its degree-k extension.
- m / n — number of transaction inputs / outputs (context-dependent; in Sec. 4.1, m = block-data length).

4.1 Data Availability Layer (P-DAL) and FRI Commitments

Heavy input tiles $A_{i,k}, B_{k,j}$ are linearly serialized into a block data vector $D = (d_0, \ldots, d_{m-1})$ over $\mathbb{F}_p$, where $p = 2^{61} - 1$. Availability rests on probabilistic Data Availability Sampling (DAS) over a ReedSolomon encoding.

ReedSolomon encoding: D is interpolated as $\varphi(X) \in \mathbb{F}_p[X]$, $\deg(\varphi) < m$, with $\varphi(\alpha_l) = d_l$, then extended with redundancy $R = 0.5$ over a domain of size $2m$:

$$D_{\text{ext}} = \big(\varphi(\beta_0), \ldots, \varphi(\beta_{2m-1})\big) \tag{19}$$

Legend:

- $D = (d_0, \ldots, d_{m-1})$ — block data vector (serialized tensor tiles).
- $\varphi(X)$ — interpolating polynomial, $\deg(\varphi) < m$, with $\varphi(\alpha_l) = d_l$.
- $\{\alpha_l\}$ — fixed base interpolation points; $\{\beta\}$ — extended evaluation domain (size $2m$).
- $R = 0.5$ — ReedSolomon redundancy factor.
- D_{ext} — RS-extended codeword.

FRI commitment: Encoding integrity is a Merkle root over the extended evaluations:

$$C_{\text{P-DAL}} = \text{MerkleRoot}\big(\varphi(\beta_0), \ldots, \varphi(\beta_{2m-1})\big) \tag{20}$$

Legend:

- $C_{\text{P-DAL}}$ — hash-based polynomial commitment (Merkle root over extended evaluations); post-quantum, no trusted setup.

Verification (DAS): A light node checks, per random sample $(\beta_z, \varphi(\beta_z))$:

1. $\text{MerklePath}(\beta_z, \varphi(\beta_z)) \to C_{\text{P-DAL}}$
2. $\text{VerifyFRI}(\pi_{\text{FRI}}, C_{\text{P-DAL}})$, a low-degree proof over a logarithmic number of folding rounds.

Legend:

- β_z — randomly sampled evaluation point during DAS.
- π_{FRI} — FRI low-degree proof.

Data-withholding resistance follows from RS redundancy: a sufficient fraction of correct samples reconstructs the block. FRI openings require $\mathcal{O}(\text{polylog } m)$ queries. While KZG offers constant-size proofs, FRI is selected to ensure strict post-quantum resistance, zero trusted setup, and structural consistency with the zk-STARK transaction layer.

4.2 Post-Quantum Transactional Model (zk-TWL)

Hash Instantiation: Let H be a STARK-friendly collision-resistant hash (e.g., Rescue-Prime or Poseidon2) with an output length in $\mathbb{F}_p^t$, where $t = 5$, ensuring $p^t > 2^{256}$ ($\geq$ 128-bit collision resistance). Digests cm, nf are multi-limb: the small field is used for proving, never for binding. Tags $\langle\cdot\rangle$ provide domain separation.

4.2.1 Value-Conservation Axiom

$$\sum_{i=1}^{m} v_i^{\text{in}} = \sum_{j=1}^{n} v_j^{\text{out}} + Fee_{tx} \pmod p \tag{21}$$

Legend:

- $v_i^{\text{in}},\ v_j^{\text{out}}$ — input / output note values (base units).
- Fee_{tx} — transfer fee (distinct from Fee_{task}, the PoUW task budget of Sec. 1.3).

Conservation is enforced entirely in-circuit. No homomorphic value commitments (e.g., Pedersen) are required, yielding a strictly curve-free and post-quantum balance mechanism. The equivalence of field reduction and integer equality is guaranteed by Eq. 27 and Eq. 28.

4.2.2 Note, Keys, Commitment, Nullifier

A note is the tuple $\eta = (v,\ \mathsf{pk},\ \rho_{\text{pre}},\ r)$.

$$\mathsf{nk} = H(\langle \text{nk} \rangle \parallel \mathsf{sk}), \qquad \mathsf{pk} = H(\langle \text{pk} \rangle \parallel \mathsf{sk}) \tag{22}$$

Legend:

- sk — secret spending key (seed).
- nk — nullifier key, one-way from sk; its secrecy gives both unforgeability and unlinkability.
- pk — public address key, committed in the note.

$$cm = H(\langle \text{cm} \rangle \parallel v \parallel \mathsf{pk} \parallel \rho_{\text{pre}} \parallel r) \tag{23}$$

Legend:

- cm — note commitment (multi-limb digest).
- v — note value; r — hiding trapdoor (chosen by the payer); ρ_{pre} — per-note seed committed in cm.

$$\rho = H(\langle \rho \rangle \parallel \rho_{\text{pre}} \parallel \mathsf{pos}), \qquad nf = H(\langle \text{nf} \rangle \parallel \mathsf{nk} \parallel \rho) \tag{24}$$

Legend:

- pos — leaf index of cm in the commitment tree at insertion.
- ρ — position-bound nullifier seed (guarantees global uniqueness even for identical notes).
- nf — nullifier, published on spend; deterministic in (nk, ρ), unlinkable from cm.

4.2.3 State: Two Append-Only Structures

The shielded state comprises two independent append-only commitments. "What exists" and "what was spent" never share a visible element, preventing spend-graph exposure.

Commitment tree $\mathcal{T}_{cm}$ (insert-only, leaves cm):

$$\text{MerklePath}(cm,\ \mathsf{pos},\ \pi_{\text{path}}) = \mathsf{CMRoot} \tag{25}$$

Legend:

- π_{path} — Merkle authentication path (membership witness).
- CMRoot — root of the insert-only commitment tree $\mathcal{T}_{cm}$ ("what exists").

Nullifier tree $\mathcal{T}_{nf}$ (indexed, leaves $\ell = (\mathsf{val}, \mathsf{next\_val}, \mathsf{next\_idx})$). Non-membership of nf is proven via a low-leaf ℓ^*:

$$\ell^*.\mathsf{val} < nf < \ell^*.\mathsf{next\_val} \tag{26}$$

Legend:

- $\ell^* = (\mathsf{val}, \mathsf{next\_val}, \mathsf{next\_idx})$ — low-leaf of the indexed nullifier tree, proving non-membership of nf.
- NFRoot — root of the indexed nullifier tree $\mathcal{T}_{nf}$ ("what was spent").

The roots CMRoot and NFRoot are committed in the header. Each spend consumes the current pair and produces the next via $\mathcal{O}(\log|\mathcal{T}|)$ updates. The structures are linked *only* through the secret nk. Set-membership ordering is enforced privately in-circuit by grand-product and lookup arguments.

4.2.4 Value Conservation and Range Limits

$$0 \le v_j^{\text{out}} < 2^{n_v} \quad \forall j \tag{27}$$

Legend:

- n_v — value bit-width ($n_v = 51$: represents $S_{\max} = 1.5 \times 10^{15}$ base units $< 2^{51}$).

$$n_v + \lceil \log_2 m \rceil < \log_2 p \qquad \text{(no-wrap invariant)} \tag{28}$$

Legend:

- No-wrap invariant: with $m \le 2^9$, $51 + 9 = 60 < 61$, so reduction mod p is the identity on honest sums $\implies$ field equality certifies integer equality.

4.2.5 Spend Relation (zk-STARK Statement)

For every input note, the Prover demonstrates knowledge of a witness satisfying $\mathcal{R}_{\text{spend}}$:

$$\mathcal{R}_{\text{spend}} : \begin{cases} cm = H(\langle \text{cm} \rangle \parallel v \parallel \mathsf{pk} \parallel \rho_{\text{pre}} \parallel r) & \text{(opening)} \\ \text{MerklePath}(cm, \mathsf{pos}, \pi_{\text{path}}) = \mathsf{CMRoot} & \text{(membership)} \\ \mathsf{pk} = H(\langle \text{pk} \rangle \parallel \mathsf{sk}) \wedge \mathsf{nk} = H(\langle \text{nk} \rangle \parallel \mathsf{sk}) & \text{(authority)} \\ \rho = H(\langle \rho \rangle \parallel \rho_{\text{pre}} \parallel \mathsf{pos}) \wedge nf = H(\langle \text{nf} \rangle \parallel \mathsf{nk} \parallel \rho) & \text{(nullifier)} \\ \ell^*.\mathsf{val} < nf < \ell^*.\mathsf{next\_val} & \text{(non-membership)} \\ \sum v^{\text{in}} = \sum v^{\text{out}} + Fee_{tx} \wedge 0 \le v_j^{\text{out}} < 2^{n_v} & \text{(conservation)} \\ cm_j^{\text{out}} = H(\langle \text{cm} \rangle \parallel v_j^{\text{out}} \parallel \mathsf{pk}_j \parallel \rho_{\text{pre},j} \parallel r_j) & \text{(outputs)} \end{cases} \tag{29}$$

Legend:

- Public statement x: CMRoot, nullifiers $\{nf_i\}$, output commitments $\{cm_j^{\text{out}}\}$, Fee_{tx}, root transition ($\mathsf{NFRoot} \to \mathsf{NFRoot}'$).
- Private witness w: $\{v_i, \mathsf{pk}_i, \rho_{\text{pre},i}, r_i, \mathsf{sk}_i, \mathsf{pos}_i, \pi_{\text{path},i}\}$ and all output openings.
- Seven clauses: opening, membership, spend authority, nullifier, non-membership, conservation+range, output notes. Spend authority binds sk inside cm — the payer (who knows r, ρ_{pre}) cannot spend.

Only the holder of sk can produce a valid nf; the payer knows r and ρ_{pre} but cannot spend. Post-quantum authority binding is evaluated inside the STARK, requiring no signature verification in the circuit.

4.2.6 Authorization as Signature of Knowledge

The shielded pool utilizes no explicit digital signatures. The STARK proof π operates as the authorization. The FiatShamir challenge commits directly to the public statement, rendering π non-malleable with respect to the state update:

$$\gamma = H\big(\langle \text{fs} \rangle \parallel \mathsf{tr} \parallel \{cm_j^{\text{out}}\} \parallel Fee_{tx} \parallel \mathsf{memo} \parallel \mathsf{nonce}\big) \tag{30}$$

Legend:

- γ — FiatShamir challenge (Signature of Knowledge binding).
- tr — STARK transcript (commitments to trace / constraint polynomials).
- memo — optional transaction memo; nonce — grinding nonce (see Eq. 44).
- π — the STARK proof itself, which *is* the authorization.

Any modification to outputs, fees, or memos invalidates π, binding the spend capability strictly to this exact transition.

4.2.7 Transparent Pool and Cross-Pool Transitions

Transparent accounts are authorized via ML-DSA (FIPS 204):

$$\sigma = \text{Dilithium.Sign}(\mathsf{sk}_D, \mathsf{tx}), \qquad \text{Verify}(\mathsf{pk}_D, \mathsf{tx}, \sigma) \in \{0, 1\} \tag{31}$$

Legend:

- $\mathsf{sk}_D, \mathsf{pk}_D$ — ML-DSA (FIPS 204) keypair for the transparent pool; σ — signature; tx — transparent transaction.

Escrow of Fee_{task}, block rewards, and public market pricing $P_{\text{compute}} = Fee_{\text{task}}/N_{\text{ops}}$ (Sec. 1.3).

$$\textbf{Shield } (T \to Z): \text{ debit } w \text{ (public)} \Rightarrow \text{append } cm \tag{32}$$

$$\textbf{Unshield } (Z \to T): \ \mathcal{R}_{\text{spend}} \text{ with revealed } w \Rightarrow \text{publish } nf, \text{ credit } w \tag{33}$$

Legend:

- w — cross-pool transition amount (public at the boundary; fixed-denomination via the reserve, Sec. 4.2.8).

4.2.8 Reserve Bridge and Economic Minimization

Value crossing pools is routed through a reserve in fixed denominations, batched over a window. The reserve backing operates as a protocol-enforced invariant:

$$B_R(t) = \sum_{\text{shield} \le t} w - \sum_{\text{unshield} \le t} w \ge 0 \tag{34}$$

Legend:

- $B_R(t)$ — reserve balance backing the shielded pool. The invariant $B_R \ge 0$ is a public solvency attestation: total unshielded $\le$ total shielded, capping catastrophic failure (no minting beyond the reserve).

$$\Delta_{\text{pool}}(t) = S_T(t) + E(t) - S_T(t{+}1) = B_R(t{+}1) - B_R(t) \tag{35}$$

Legend:

- Δ_{pool} — net cross-boundary flow per epoch (the only quantity the bridge exposes; a feature, not a leak).
- $S_T(t)$ — total transparent-pool supply at epoch t (public by auditability).

- $E(t)$ — scheduled emission into the transparent pool at epoch t (public).

The reserve $B_R(t)$ acts as a continuous solvency attestation. Δ_{pool} is the sole quantity exposed by the bridge: the net cross-boundary flow. As a difference of public balances, it is mathematically unavoidable and serves as public proof of non-inflation.

The internal shielded graph remains mathematically isolated from Δ_{pool}. A well-formed bridge additionally hides the deposit-withdrawal pairing, identity, and amount beyond its denomination class:

$$k = \big|\{\text{crossings of denomination } D \text{ in window } W\}\big|, \qquad \Pr[\text{link}] \le 1/k \tag{36}$$

Legend:

- k — boundary anonymity-set size; D — denomination class; W — aggregation window (blocks).

Boundary leakage is isolated via economic minimization: miner rewards are born shielded, internal transfers are shielded-by-default, and bridge flushing operates according to strict batching parameters:

$$D \in \{1, 10, 100, 1000\}; \qquad \text{flush} \iff (k \ge k_{\min}) \vee (\Delta_{\text{blocks}} \ge W) \tag{37}$$

Legend:

- D — denomination rungs (TKN), sized for miner-scale payouts.
- $k_{\min} = 64$ — minimum anonymity set before flush ($\Pr[\text{link}] \le 1/64$); $W = 128$ — latency ceiling in blocks; Δ_{blocks} — blocks elapsed since batch open.

Born-shielded rewards: The coinbase mints the block reward directly as a shielded note to the miner's key, bypassing the bridge entirely:

$$cm_{\text{coinbase}} = H(\langle \text{cm} \rangle \parallel R_{\text{block}}(t) \parallel \mathsf{pk}_{\text{miner}} \parallel \rho_{\text{pre}} \parallel r) \tag{38}$$

Legend:

- $R_{\text{block}}(t)$ — block reward in epoch t. Its value is initially public (emission is deterministic, Sec. 1.2); privacy emerges only after the first shielded spend mixes the amount into the graph. This is the accepted coinbase → shielded trade-off: the reward joins the anonymity set at mint, but is not itself hidden until moved. The mechanism grows the internal anonymity set with every block at no bridge cost.

4.3 Block Structure and State Integration

$$CID = \text{SHA} - 256(\mathsf{MatrixA} \parallel \mathsf{MatrixB}) \tag{39}$$

Legend:

- CID — content identifier in the availability layer; $\mathsf{MatrixA}, \mathsf{MatrixB}$ — client task data.

$$\mathsf{Header} = H\big(\mathsf{PrevHash} \parallel \mathsf{CMRoot} \parallel \mathsf{NFRoot} \parallel \mathsf{TPRoot} \parallel \mathsf{RootPoUW} \parallel \mathsf{MinerID} \parallel \mathsf{VDFout} \parallel \mathsf{Timestamp}\big) \tag{40}$$

Legend:

- $\mathsf{PrevHash}$ — previous block descriptor.

- CMRoot / NFRoot — commitment / indexed-nullifier tree roots (shielded exists / spent).
- TPRoot — Merkle root over transparent-pool balances.
- RootPoUW — root over scientific-matrix references and SMA evaluations (binding scheme, Sec. 2.4).
- MinerID — block producer's cryptographic identifier; VDFout — VDF output (time anchor); Timestamp — deterministic closure time.

$$\pi_{\text{block}} \vdash \Big[\bigwedge_{\text{spends}} \mathcal{R}_{\text{spend}}\Big] \wedge (\mathsf{CMRoot}, \mathsf{NFRoot})\text{-transition} \wedge \Big[\bigwedge_{\text{shares}} \text{SMA}\Big] \tag{41}$$

Legend:

- π_{block} — single recursive STARK certifying, simultaneously, all shielded spends (Eq. 29), both root transitions, and all PoUW shares (SMA, Sec. 2.2).

Verification cost is strictly $\mathcal{O}(1)$ roots + π_{block} + Dilithium signatures + VDF pair, while each shielded spend requires only $\mathcal{O}(\log|\mathcal{T}|)$ constraints.

4.4 Parameter Selection and Cryptographic Hardness Bounds

The configuration mandates $p = 2^{61} - 1$, a trace length $n \leq 2^{24}$, a rate $\rho = 1/8$, and a target of 128-bit quantum security, structured strictly around proven loss boundaries.

Proximity gap conjectures bounded by capacity $(1 - \rho)$ have been formally disproven over prime fields. PROTOCOL parameterizes strictly within the proven Johnson regime $(1 - \sqrt{\rho})$. Meeting the 128-bit security parameter within this regime requires the field term $n/|\mathbb{F}|$ to be negligible. For a base field $p \approx 2^{61}$, the quadratic extension $\mathbb{F}_{p^2}$ yields $\sim 2^{-98}$ (insufficient), forcing evaluation over $\mathbb{F}_{p^3}$:

$$\frac{n}{|\mathbb{F}_{p^3}|} \leq 2^{-159} \tag{42}$$

Legend:

- n — trace length ($\leq 2^{24}$); k — FRI extension degree. $k = 3$ gives field term 2^{-159}; $k = 2$ (2^{-98}) is insufficient. FRI folds/challenges use $\mathbb{F}_{p^3}$; PoUW/commitment arithmetic stays in $\mathbb{F}_p$.

Shielded integrity rests on the knowledge-soundness of the Fiat-Shamir compiled argument. Naive exponentiation loss Q^μ is replaced by the proven Measure-and-Reprogram 2.0 framework (DFMS), bounding the QROM error to a quadratic loss $\mathcal{O}(q^2)$:

$$\varepsilon_{\text{FS}} \leq (Q+1)\kappa \quad \text{(ROM)}; \qquad \varepsilon_{\text{FS}}^{\text{QROM}} \leq c \cdot q^2(Q+1)\kappa \quad \text{(QROM)} \tag{43}$$

Legend:

- ε_{FS} — FiatShamir knowledge error.
- κ — round-by-round knowledge error of the underlying AIR+FRI IOP.
- Q — classical random-oracle queries (AFK bound); q — quantum oracle queries (DFMS bound); c — small constant.

To clear the 128-bit quantum margin (accounting for Grover):

$$1.5t + \frac{g}{2} \geq 256 \implies (t, g) = (160, 32) \tag{44}$$

Legend:

- t — number of FRI queries; g — grinding bits ($g = 32 \to 16$ quantum bits via Grover). 1.5 = bits/query in the Johnson regime at rate $\rho = 1/8$. Target: 128-bit quantum with the DFMS $\mathcal{O}(q^2)$ loss absorbed.

Where $t = 160$ queries and a transcript grinding factor of $g = 32$ forces 2^{32} prover work per block (verified instantly), securely absorbing the $\mathcal{O}(q^2)$ QROM lift without theoretical gaps.

4.4.1 Proof Size Estimate

Reference values for a 2-input / 2-output spend. These are engineering estimates, not a measurement; the exact figure depends on the concrete prover (hash layout, column packing, Merkle path deduplication can move it by ±30%). The order of magnitude is firm.

AIR parameters:

$$N_{\text{rows}} = 2^{14}, \quad w_{\text{cols}} = 68, \quad D_{\text{deg}} = 8, \quad \#\text{hash} \approx 150 \tag{45}$$

Legend:

- $N_{\text{rows}} = 2^{14}$ — trace length (active $\sim 2^{13.5}$, padded to a power of two). Dominated by Merkle paths: depth $d = 32\times$ two paths per input (membership + non-membership).
- $w_{\text{cols}} = 68$ — Poseidon2 state (width 16) + selectors + range-check + lookup permutation columns.
- $D_{\text{deg}} = 8$ — maximum constraint degree (S-box); consistent with blowup $\times 8$ (blowup $\geq D - 1$).
- Hash split — Poseidon2 (width 16) for the trace bulk; Keccak only on value-binding rows $(cm,\, nf)$.

Proof-size breakdown (field elements in $\mathbb{F}_p \approx 8$ B; FRI-fold values in $\mathbb{F}_{p^3} \approx 24$ B; FRI Merkle nodes 16 B; folding factor $4 \to 6$ rounds):

$$\underbrace{t(w_{\text{cols}} D_{\text{deg}} + 17 \cdot 16)}_{\text{trace}\approx 130\text{ KB}} + \underbrace{t \cdot 6 \cdot (4 \cdot 24 + 12 \cdot 16)}_{\text{FRI layers}\approx 275\text{ KB}} + \underbrace{\text{roots, OOD, final poly, PoW}}_{\approx 20\text{ KB}} \approx 300\text{–}400\text{ KB} \tag{46}$$

With $t = 160$ (Eq. 44) the query count dominates. The proven Johnson+quantum regime uses $\sim 2\times$ the queries of a conjecture-based system (~ 80), so the proof is $\sim 2\times$ larger — a deliberate cost of certainty. Deduplicating shared upper tree nodes brings the figure toward the lower end (~ 300 KB).

Recursion changes the economics. Eq. 46 is the aggregating prover's cost, not the chain's. On-chain there is one π_{block} (Eq. 41), of the same order ($\sim$ 300–500 KB) independent of the number of transactions in the block. Per-transaction on-chain cost is therefore amortized to zero.